D | 外部依赖可靠性

合作方已经创建维修案件，回复却在途中丢失。本地程序只看见一次超时：现在说“失败”，可能与对方已经完成的事实冲突；再发一次，又可能安排两次维修。

外部依赖把问题从“我的函数算得对不对”推进到“另一套系统做了什么，我有什么证据”。本册学习怎样识别失败、限制等待、保护重复传输，并在不能确认时留下准确状态和下一处理者。它们共同决定：程序可以向用户承诺到哪一步。

第一次只看这里

直接进入 D01。第一次只追一次受控调用：成功或失败发生在哪一层，本地程序怎样保持诚实。

本册的详细关系图、章节路线和查用说明放在全书后部。遇到‘这一步和前后哪里相连’时再回看，不需要在第一轮先读完。

目录

第一次只看这里 1

D01 | 先查清失败发生在哪里 3

D02 | 每次再试，都要花同一份预算 9

D03 | 回复丢了，怎样仍只做一遍 14

D04 | 不能确认时，怎样把事情接住 23

D05 | 把调用、效果与本地状态放在一起验证 27

全书收束 | 外部依赖可靠性的知识关系总览 35

来源与查用入口 37

本册总览（需要时再回看） 39

本册知识结构总览 40

D01 | 先查清失败发生在哪里

本章先判断：失败发生在本地输入、请求交接、对方响应，还是本地解析；不同位置决定我们应当修正、重试、停止还是转人工。 先观察一次成功，再只改变一项条件。

用户询问 LOCAL-0042 所在邮编是否支持上门服务。页面没有拿到答案，可能是用户没填邮编，也可能是合作方返回错误，还可能是回复超时。它们都表现为“这次没查成”，下一步却完全不同。

先让一次成功完整走通，再只改变外部返回的东西。这样就能看见：程序在哪一步停止，哪些事实已经存在，哪些还不知道。

一次查询里有两份不同的回复

本地接口接受 `GET /cases/LOCAL-0042/serviceability?postal_code=200000`。本地程序再向合作方发出 `GET /serviceability?postal_code=200000`。合作方成功时返回：

```json
{"serviceable": true, "zone": "east"}
```

`serviceable` 必须是布尔值，`zone` 必须是非空文字。这里确认的是服务范围；没有创建案件，也没有占用预约名额。

请求方向

用户 → 本地接口 → 合作方查询

回复方向

用户 ← 本地回复 ← 合作方回复

第二份回复不会自动原样成为第一份。本地程序要检查合作方结果，再形成自己的公开结果。负责构造合作方请求、检查回复、翻译结果的函数，称为 adapter，即适配边界。它让业务代码取得明确的结果或失败类别，不必在每处重复理解外部协议。

邮编 200000 通过本地检查后，adapter 发出请求，得到合作方 200，解析 JSON，检查两个字段，交回服务范围结果；本地再加入 local_case_id，向用户返回自己的 200。

让合作方按照指定剧本回答

等真实合作方偶尔出错，很难验证每条分支。现在把 HTTPX 发出的 request 交给一个本地函数，由它返回预定 response 或抛出预定异常。这就是 `MockTransport` 的作用。HTTPX 仍然构造真实的 request 对象，adapter 仍然执行；被替换的是传输位置。HTTPX Transports

本地接口使用 `TestClient` 进入，不启动监听端口。它执行本地路由、输入校验和 response 生成，适合验证应用合同。FastAPI Testing

五种失败为何不能用一个“重试”处理？

输入或剧本	最早偏离	合作方调用	合作方 response	本地 response
缺邮编	本地输入校验	0	未发起调用	422
500	对方 HTTP 状态	1	已有，状态 500	502
timeout	发送或等待	1	本次调用未交回可用 response	504
200，非 JSON	内容解析	1	已有，状态 200	502
200，字段类型错	内容合同	1	已有，状态 200	502

前四列解释失败发生在哪、已有何种证据，最后一列是本地对用户的表达。内容解析与字段验证归在同一类内容失败，因而得到四个主要位置：本地输入、传输、HTTP 状态、内容合同。

缺少邮编时，FastAPI 在进入 endpoint 前就拒绝输入。实验用 calls 列表记录进入外部替身的每次调用，`calls == []` 证明外部边界根本没被触及；不能因为页面也有一个错误状态码，就让合作方承担错误。

500 路线已经有合作方 response，adapter 可以读取其状态。timeout 路线中，`response = partner.get(...)` 没有正常返回，左侧 response 没有取得值。本地接口仍能给最初用户一个 504，所以“用户收到了 response”不证明“本地收到了合作方 response”。这里模拟完整请求调用没有交回 response；真实流式读取还可能在已收到头部后读正文超时，应按具体阶段保留证据，不能抹去已经收到的信息。

坏 JSON 与坏字段又不同：两者都有 HTTP 成功状态，但前者连内容都不能解析，后者已经取得 Python 对象，只是它不符合用途所需结构。把坏字段默认为 False 会让“无法判断是否支持服务”变成“确认不支持”，悄悄改变了事实。

> **第一次学到这里先停。** 只记住“先找最早偏离”：本地输入没通过就没有外部调用；500 已有回应但不成功；timeout 没有可用回应；坏 JSON 或坏字段是在内容边界失败。重试、幂等和人工接管，都是在这张失败地图上决定下一步，不需要现在一起背。

替换哪一处，决定测试证明到哪里

如果把 check_serviceability 整个换成预设字典，只能看到 endpoint 如何包装字典，不能证明参数、status 或字段检查。当前替换传输，恰好保留本章要观察的这些步骤。

有状态 fake 会更进一步：保存创建过的结果，支持查询和重复调用。它与 MockTransport 可以配合，一个规定业务行为，一个把 HTTPX request 交给它；不是必须二选一。

相应地，这些测试没有经过真实 DNS、TLS、网络或供应商。进程内测试不证明端口能连，预设成功也不证明实际凭据有效。以后接入真实服务时，受控集成测试增加实际环境证据，仍需要这些稳定失败测试保护本地分支。

运行时最值得核对的一项变化

准备实际运行时，先只查 D05 的“先复制，再在实验目录运行”：复制 `partner-case`，建立 `.venv`、安装依赖并完成 `pip check` 后回到这里。暂不运行那里后面的 `pytest`、`milestone_check.py` 或恢复脚本，也不需要先读完 D05 的场景矩阵。

下面程序完整保存为实验副本中的 `d01_serviceability.py`。先读 check_serviceability，再读 build_app，最后看 run_case 如何把两边接起来。

PYTHON

```
import httpx
from fastapi import FastAPI, HTTPException, Query
from fastapi.testclient import TestClient


class PartnerFailure(Exception):
    def __init__(self, category, provider_status=None):
        super().__init__(category)
        self.category = category
        self.provider_status = provider_status


def check_serviceability(partner, postal_code):
    try:
        response = partner.get(
            "/serviceability", params={"postal_code": postal_code}
        )
    except httpx.TimeoutException as error:
        raise PartnerFailure("provider_timeout") from error
    except httpx.RequestError as error:
        raise PartnerFailure("provider_transport_error") from error

    try:
        response.raise_for_status()
```

PYTHON · 续

```
    except httpx.HTTPStatusError as error:
        raise PartnerFailure("provider_bad_status", response.status_code) from
          error

    try:
        body = response.json()
    except ValueError as error:
        raise PartnerFailure("provider_bad_content", response.status_code)
          from error
    if not isinstance(body, dict):
        raise PartnerFailure("provider_bad_content", response.status_code)
    serviceable, zone = body.get("serviceable"), body.get("zone")
    if (
        not isinstance(serviceable, bool)
        or not isinstance(zone, str)
        or not zone.strip()
    ):
        raise PartnerFailure("provider_bad_content", response.status_code)
    return {"serviceable": serviceable, "zone": zone.strip()}


def build_app(partner):
    app = FastAPI()

    @app.get("/cases/{local_case_id}/serviceability")
    def serviceability(
        local_case_id: str, postal_code: str = Query(min_length=1)
    ):
        try:
            result = check_serviceability(partner, postal_code)
        except PartnerFailure as error:
            status = 504 if error.category == "provider_timeout" else 502
            raise HTTPException(status, error.category) from error
        return {"local_case_id": local_case_id, **result}

    return app


def run_case(mode, include_postal_code=True):
    calls = []

    def handler(request):
        calls.append((
            request.method, request.url.path,
            request.url.params.get("postal_code")
        ))
```

PYTHON · 续

```
        if mode == "timeout":
            raise httpx.ReadTimeout("synthetic timeout", request=request)
        if mode == "500":
            return httpx.Response(500, json={"detail": "synthetic_error"})
        if mode == "bad_json":
            return httpx.Response(200, text="not JSON")
        if mode == "bad_field":
            return httpx.Response(
                200, json={"serviceable": "yes", "zone": "east"}
            )
        if mode == "success":
            return httpx.Response(
                200, json={"serviceable": True, "zone": "east"}
            )
        raise AssertionError("unknown fake mode")

    with httpx.Client(
        base_url="https://partner.invalid",
        transport=httpx.MockTransport(handler),
        timeout=1.0,
        trust_env=False,
    ) as partner:
        with TestClient(build_app(partner)) as local:
            params = {"postal_code": "200000"} if include_postal_code else {}
            response = local.get(
                "/cases/LOCAL-0042/serviceability", params=params
            )
    return response, calls


response, calls = run_case("success")
assert response.status_code == 200
assert response.json() == {
    "local_case_id": "LOCAL-0042", "serviceable": True, "zone": "east"
}
assert calls == [("GET", "/serviceability", "200000")]

response, calls = run_case("success", include_postal_code=False)
assert response.status_code == 422 and calls == []

for mode, status, reason in [
    ("500", 502, "provider_bad_status"),
    ("timeout", 504, "provider_timeout"),
    ("bad_json", 502, "provider_bad_content"),
    ("bad_field", 502, "provider_bad_content"),
]:
    response, calls = run_case(mode)
```

PYTHON · 续

```
    assert response.status_code == status
    assert response.json() == {"detail": reason}
    assert len(calls) == 1
print("D01: success and five controlled failures passed")
```

PartnerFailure 把失败类别带回本地接口；它没有恢复业务成功。`raise` 中断当前函数，控制转到匹配的 `except`。这里 timeout 先于一般 RequestError 捕获，因为 timeout 属于后一类中的具体情况，先捕获大的类别会失去这份区别。

`response.raise_for_status()` 在已经取得 response 后检查状态；`response.json()` 把内容变成 Python 值；后面的字段判断决定这个值能不能用于本次查询。JSON 解析通过但 serviceable 的值为文字 "yes"，仍然不能成为约定的布尔结果。

在准备好的实验副本中运行：

POWERSHELL

```
& .\.venv\Scripts\python.exe .\d01_serviceability.py
if ($LASTEXITCODE -ne 0) { throw 'D01 实验未通过' }
```

最后应显示 `D01: success and five controlled failures passed`。其中包含同类内容失败的两种原因，不是把五种失败当作五个架构层。

再看一个有意错误的记录：本地 422，外部调用 1 次，外部新增结果 0 个。最早错误是外部调用数；输入已经被拒绝，不应继续调用。没有新增结果并不能证明调用从未发生。能指出这一点，比记住五个测试名称更有用。

下一步才是：已知失败在哪，是否值得再试，以及最多还能等多久。

D02 | 每次再试，都要花同一份预算

合作方在 east 区的服务范围已经明确，但还不知道 2026-09-03 有多少可预约时段。这里查询 available_slots，只读取容量，不创建案件或保留名额。

一次查询超时后，再读一次可能成功。但用户等的是整件事：第一次等待、两次之间的暂停、第二次等待都会计入。只写“每次最多一秒”，没有回答用户总共要等多久。

把总共允许等待的时间先写出来

本章采用一份便于计算的合成政策：总预算 2.5 秒，每次上限 1 秒，两次之间等 0.2 秒，最多尝试 2 次。最多 2 次包含第一次，不是“第一次以外再重试两次”。

从开始时刻算出停止点，称为 deadline。每次发起调用前，重新看还剩多少时间：

```
deadline = started + 2.5
remaining = deadline - now
allowed = min(1.0, remaining)
```

allowed 是这一次能使用的上限。失败以后主动等待的 0.2 秒叫 backoff；它让两次调用不紧贴在一起，也要从 remaining 里支付。

假设第一次耗时 1 秒后超时，第二次只需 0.1 秒成功：

动作	开始时刻	结束时刻	发生了什么
第一次查询	0.0	1.0	超时，尚无容量结果
backoff	1.0	1.2	剩余 1.5 秒足以支付等待
第二次查询	1.2	1.3	收到合法容量 3

总共消耗 1.3 秒，第二次调用仍可分配 1 秒。若两次都超时，则消耗 2.2 秒；虽然余额还有 0.3 秒，次数已经用完，仍要停止。如果总预算改成 1.1 秒，第一次超时后仅剩 0.1 秒，连 backoff 都付不起，不应先等 0.2 秒再决定。

预算使用单调时钟（monotonic clock）计算经过时间。墙上的日期、时区或校时可能改变，不能因此延长或缩短某次操作的余额。

哪些失败值得花第二次机会？

这里规定只有 timeout 可以进入再次尝试；401、其他错误状态、坏 JSON 或错误字段立即停止。不是宣称这些类别在所有服务里都永久不变，而是当前容量接口没有声明更宽的恢复合同。

401 通常要求检查凭据或权限，原封不动重复不能修复。字段从整数 3 变成文字 three，要求核对内容合同。500 虽然来自服务器，也不能仅凭数字自动判断它短暂、安全且适合重放。

再试要同时满足三个事实：当前失败可能暂时发生；这个操作可安全重复；时间和次数仍够。本章选择只读容量查询，是因为它不请求创建任何业务对象。方法名是 GET 仍要有符合读取语义的实际实现；不能把会创建预约的错误实现因为名字叫 GET 就当成安全。RFC 9110 的安全与幂等方法

后面的创建案件配套会明确允许 timeout 和 500/502/503/504 在稳定 key 保护下有限重试。那是一份不同、显式的合成协议，不能反过来把这里未声明的 500 当作可重试。

用一个可推进的时钟看见控制路线

下面完整程序保存为实验副本中的 `d02_capacity_budget.py`。它不访问网络，也不真的暂停。fake clock 的时间就是一个数，sleep 只把它增加；sender 按列表依次制造结果。这样既能检查时间，也能证明某一条预定结果根本没有被消费。

PYTHON

```
class AttemptTimeout(Exception):
    pass


class CapacityUnavailable(Exception):
    pass


class FakeClock:
    def __init__(self):
        self.now = 0.0
        self.sleeps = []

    def monotonic(self):
        return self.now

    def sleep(self, seconds):
        self.sleeps.append(seconds)
        self.now += seconds


def query_capacity(send, clock, events, total=2.5):
    deadline = clock.monotonic() + total
    for attempt in range(1, 3):
        remaining = deadline - clock.monotonic()
        if remaining <= 0:
            raise CapacityUnavailable("budget_exhausted")
        allowed = min(1.0, remaining)
```

PYTHON · 续

```
        started = clock.monotonic()
        try:
            status, body = send(allowed)
        except AttemptTimeout as error:
            events.append((
                attempt, round(clock.monotonic() - started, 3), "timeout"
            ))
            if attempt == 2:
                raise CapacityUnavailable("attempt_limit") from error
            if deadline - clock.monotonic() <= 0.2:
                raise CapacityUnavailable("budget_exhausted") from error
            clock.sleep(0.2)
            continue

        if clock.monotonic() > deadline:
            raise CapacityUnavailable("late_response")
        if status != 200:
            raise CapacityUnavailable(f"non_retryable_status_{status}")
        value = body.get("available_slots") if isinstance(body, dict) else None
        if not isinstance(value, int) or isinstance(value, bool) or value < 0:
            raise CapacityUnavailable("bad_content")
        events.append((
            attempt, round(clock.monotonic() - started, 3), "success"
        ))
        return value
    raise AssertionError("unreachable")


def run(outcomes, total=2.5):
    queue = list(outcomes)
    clock, calls, events = FakeClock(), [], []

    def send(allowed):
        calls.append(round(allowed, 3))
        outcome = queue.pop(0)
        if outcome == "timeout":
            clock.now += allowed
            raise AttemptTimeout()
        clock.now += 2.6 if outcome == "late" else 0.1
        if outcome in ("success", "late"):
            return 200, {"available_slots": 3}
        if outcome == "bad_content":
            return 200, {"available_slots": "three"}
        return int(outcome), {}

    try:
        result, error = query_capacity(send, clock, events, total), None
```

PYTHON · 续

```
    except CapacityUnavailable as failure:
        result, error = None, str(failure)
    return {
        "result": result, "error": error, "allowed": calls,
        "elapsed": round(clock.now, 3), "events": events,
        "sleeps": clock.sleeps, "unused": len(queue),
    }


first = run(["success"])
assert first["result"] == 3 and len(first["allowed"]) == 1
recovered = run(["timeout", "success"])
assert recovered["result"] == 3 and recovered["elapsed"] == 1.3
assert recovered["allowed"] == [1.0, 1.0]
assert recovered["sleeps"] == [0.2]
stopped = run(["timeout", "timeout"])
assert stopped["error"] == "attempt_limit" and stopped["elapsed"] == 2.2
short = run(["timeout", "success"], total=1.1)
assert short["error"] == "budget_exhausted" and short["unused"] == 1
assert short["sleeps"] == []
for outcome in ["401", "500", "bad_content"]:
    failed = run([outcome, "success"])
    assert failed["result"] is None and len(failed["allowed"]) == 1
    assert failed["unused"] == 1
assert run(["late"])["error"] == "late_response"
print(recovered)
print("D02: bounded policy passed")
```

`continue` 是超时后回到下一轮的唯一入口。其他失败直接抛出异常，没有悄悄消费列表中后面的 success。内容检查排除 bool，因为 Python 的 True 也满足 isinstance(True, int)；业务容量要的是整数数量，不能让布尔值冒充一个名额。

在实验副本运行 `python d02_capacity_budget.py`，应先看到结果 3、两次 allowed、elapsed 1.3 及两个事件，再看到通过文字。这里的 sender 已直接给出解析后的内容，D01 的 JSON 解析没有重复实现；它专门验证预算与决策。

改变总预算为 1.25 秒，用相同 timeout→success 剧本，可以预测第二次 allowed 只有 0.05 秒左右。这个 fake 故意允许 success 消耗 0.1 秒，因此返回时超出 deadline，得到 late_response。这暴露了另一个重要保护：不仅调用前看余额，收到结果后也要判断是否已经过期。

一个 timeout 数字为什么不是整项工作的硬秒表？

HTTPX 分别提供 connect、read、write、pool 等等待限制：建连、等一块数据、发一块数据、从连接池取得连接。`httpx.Timeout(1.0)` 并不等于“这个业务动作不论经过多少阶段，总共恰好一秒后强制结束”。HTTPX Timeouts

应用维护 deadline，能控制是否发起下一次、是否支付 backoff、是否按正常成功接纳过晚结果；但在同步调用尚未返回时，循环外面的时钟判断不能凭空打断它。真实系统若要求严格的端到端取消，还需要运行环境的取消机制，并验证被取消时外部效果怎样处理。fake 主动推进到 2.6 秒，只证明迟到结果如何被本地分类。

同样，最多两次应当只有一处负责。若业务层做两轮、底层每轮又最多尝试两次（含首次），外部实际可能被调用四次。当前 sender 只发送一次，策略层决定是否再次调用，日志中的 attempt 才与实际次数一致。

事件记录 attempt、耗时和结果类别，足以解释是否重试；不需要复制 token 或整个回复。大规模同时重试可能需要随机抖动（jitter）分散时间，但抖动和 backoff 一样消耗总预算，不能成为延长承诺的理由。

要真正理解本章，应能解释“还有时间为什么仍停止”和“允许重试为什么仍没有第二次”。前者可能用完次数，后者可能付不起等待，或操作本身不可安全重复。下一章要解决的，正是创建案件这种有副作用的重复风险。

D03 | 回复丢了，怎样仍只做一遍

先只看同一 intent、同一不可变 payload 和同一 key 的重放；目标是确认一次业务效果不会被做两遍。

现在真正为 LOCAL-0042 创建合作方案件。第一次发送以后，对方已经保存 P-0042，却没有把回复成功交到本地。只看本地的 timeout，无法判断对方有没有做。

这里的副作用（side effect）就是新增一条合作方案件。我们允许请求再传一次，但不希望同一次派单因此产生两条案件。要做到这一点，两边必须共同识别：“这是先前那件事的再次传输。”

先看两边各自留下了什么

把本次动作叫作 `OUT-0042-V1`。它表示“按这一份确定内容，为 LOCAL-0042 创建案件”，不是工单本身。由它生成稳定的幂等键（idempotency key）：

```
partner-case:OUT-0042-V1
```

本章自包含 provider 约定：同 key、同业务内容再次到达时返回原结果；同 key、不同内容时拒绝冲突。它先把 key、输入和案件结果一起持久保存，再回复。header 名、保存期限和冲突格式属于具体 provider 的合同，不能只给任意 POST 加这个 header 就宣称获得保护。

假设第一次发送后回复丢失，两边会出现这样的状态：

位置	已保存的事实	当前不知道什么
本地	OUT-0042-V1、原输入、稳定 key、pending	对方到底有没有完成
合作方	这个 key 已对应原输入和 P-0042	本地是否收到了回复

timeout 只能改变本地获得的证据，不能擦掉对方已经提交的 P-0042。 用这个区别再看一次重传：

第一次：

K + 原输入 → 创建 P-0042 → 提交 → 回复丢失

第二次：

K + 原输入 → 找到 P-0042 → 返回原结果

观察到的结果

外部调用次数：2

外部业务效果：1

本地状态：取得并校验结果后，才变为 partner_confirmed

这就是本章的幂等关系：重复传输同一意图，预期业务效果不因此增加。它不是“每次 response 完全相同”；第一次可能 201，重复返回可能 200，关键是两次都指向原来的业务效果。

为什么第一次调用以前就要保存完整意图？

若 key 只存在内存里，进程退出就丢失；稍后的重试可能生成新 key。若只保存 key，输入却每次从可编辑表单重建，两次传输又可能暗中改变内容。因此第一次发送前，本地先提交这组信息：

字段	示例含义
intent_id	OUT-0042-V1，区分一次稳定外部动作
local_case_id	LOCAL-0042，说明动作属于哪个本地案件
payload_contract_version	partner-case-v1，说明输入采用哪版字段规则
canonical_payload_json	规范化后冻结的业务 JSON，供重新传输
payload_digest	上述 UTF-8 字节的 SHA-256 摘要，用于比较
idempotency_key	partner-case:OUT-0042-V1，供合作方识别同一意图
state	pending，目前尚未确认

规范化（canonicalization）先把同一业务内容转成一种确定表示。例如字典的键顺序可能不同，JSON 空格也可能不同；本例统一按键排序并去掉多余分隔空格，再计算摘要。下面两份输入的键顺序不同，但规范化结果相同：

```
{"priority": 2, "category": "hardware"}
{"category":"hardware","priority":2}
```

SHA-256 产生固定长度的摘要。它适合检查保存的输入与当前输入是否一致，不能还原输入，也不是加密保存。真实地址或摘要若在 canonical JSON 中，仍需控制谁能读取。

这里的规范化只服务当前字段合同，不宣称是所有语言与数据类型通用的 JSON 规范；日期、浮点数、Unicode 或默认值的处理发生变化时，必须重新约定。合同版本让“这个字符串按什么规则得到”也可追踪。

同一轮有限重试都从这份冻结输入发送。同一个 intent 不得在第二次尝试前悄悄换成表单刚改的新内容。摘要也不应随尝试重新覆盖，否则原始意图的证据会消失。

新的一次请求，不一定是新的一件事

request ID 关联一次运行，例如 R-501 的入口、两次 attempt 和最终日志。用户稍后再次提交，同一业务意图可以由新的 R-502 处理，但仍使用原 intent、原输入和原 key。

相反，用户把打印机维修换成账户问题，是业务输入发生变化。即使仍然属于 LOCAL-0042，也需要新 intent `OUT-0042-V2`、新 canonical JSON、新摘要和新 key。旧意图保留，不能覆盖成新内容。新 key 允许产生新的业务效果；若旧效果需要取消，那是另一个经过确认的动作，新建意图本身不会取消旧案件。

跨请求遇到旧 pending 时，要区分三种情况：

找到的本地记录	可以继续什么
当前合同版本、完整 canonical 输入，且当前业务摘要相同	才有资格按原 key 进入显式、有限的受控重放
只有摘要，缺输入或合同版本	停止自动发送，交人工核对；摘要不能重建请求
当前业务摘要已变	拒绝把新内容塞给旧 intent；经确认的新业务使用新 intent/key

“具备重放材料”不等于后台看见 pending 就一律自动重发。真实 provider 还可能有 key 有效期、并发限制和查询入口，必须服从它们；资料不足时保持未知。D05 的实验由一次新的本地 POST 显式触发恢复，不在 GET 读取时发送。

为什么“有一个 key”还不够？

错误实现若在每个 attempt 中生成新 key，合作方会合理地把两次请求当成两件事：第一次创建 P-0042 后丢回复，第二次创建 P-0043。最后 HTTP 成功、本地只记 P-0043，并不能抹去第一条案件。应在建立 intent 时生成一次并持久保存 key，所有传输读取它。

另一种错误只保存“已发送”标志。这个标志既可能对应“对方没收到”，也可能对应“对方已提交但回复丢失”；它没有足够信息决定是否再次创建。稳定 key 与对方保存的结果关系，才给重传提供依据。

本地事务保护 intent 和本地状态，provider 事务保护它自己的 key 与结果；任何一方的 rollback 都不能自动撤回另一方。不要把漫长网络等待放进本地写事务，误以为两边因此一起成功或一起失败。通常先提交 pending，离开本地事务后调用，取得可用结果再用短事务确认。

这份程序演示顺序执行的恢复。它不证明所有并发、突然断电、key 过期或跨进程竞争已经解决。真实 provider 应当用唯一约束和事务保护 key 对应关系，并说明竞争、保留期限与查询方式。需要记住的是这份责任，完整实现要随目标系统核对。

HTTP 的 safe methods、PUT、DELETE 具有相应的幂等语义；POST 不能仅凭方法名称自动安全重放。本章 POST 的保护来自额外的 provider 合同。RFC 9110 第 9.2.2 节

如果远端成功了，本地写入却失败呢？

provider 已创建 P-0042，随后本地更新遇到 SQLite 错误，是另一个两边分离的窗口。此时既不能声称本地 confirmed 已提交，也不能生成新 key 补做一次。先保留之前完整的 pending 意图；下一次显式恢复核对同一输入、版本和 key，由 provider 返回原结果，再尝试本地确认。

这时外部调用数可能增加，本地确认次数却仍只成功一次；provider 效果仍应是一次。D05 的组合实验把这种恢复与完整 HTTP 路线一起核对。若连本地失败记录也写不进去，应明确暴露本地存储故障，不能虚构一个已保存的人工队列。

两份临时数据库怎样把这个过程留下来？

下面完整程序保存为实验副本中的 `d03_idempotency_demo.py`。它采用两份独立 SQLite 文件，各自提交；provider 是本地函数，没有 HTTP 或真实网络。本地请求与 provider payload 使用本章列出的合成字段，编号 P-0042 属于这个独立程序。

程序故意直接演出两次调用，而不再实现 D02 的时间循环。它的证据集中在“发送前持久化、回复丢失、同 key 恢复、输入冲突、旧记录不能盲发”。

读这段程序时先只跟三条线：`save_intent` 把不可变输入和 key 写进本地，`create_case` 在 provider 侧提交效果，`confirm` 再把已取得的编号写回本地。第一次 `lose_response=True` 只丢回复，不撤销 provider 的提交；第二次调用应沿同一 key 找回原编号。代码末尾再看两种边界：改输入会冲突，缺少冻结输入的旧 pending 会在本地停住。这样每个断言都能对应到一份状态或一次外部效果。

PYTHON

```
import hashlib
import json
import sqlite3
from contextlib import contextmanager
from pathlib import Path
from tempfile import TemporaryDirectory

VERSION = "partner-case-v1"


class ResponseLost(Exception):
    pass


class Conflict(Exception):
    pass


def canonical(payload):
    return json.dumps(
        payload, ensure_ascii=False, sort_keys=True, separators=(",", ":")
    )


def digest(text):
    return hashlib.sha256(text.encode("utf-8")).hexdigest()


@contextmanager
def connect(path):
    db = sqlite3.connect(path)
    try:
        yield db
        db.commit()
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
```

PYTHON · 续

```
def setup(local, provider):
    with connect(local) as db:
        db.execute("""
            CREATE TABLE intents (
                intent_id TEXT PRIMARY KEY NOT NULL,
                local_case_id TEXT NOT NULL,
                payload_contract_version TEXT,
                canonical_payload_json TEXT,
                payload_digest TEXT NOT NULL,
                idempotency_key TEXT UNIQUE NOT NULL,
                state TEXT NOT NULL,
                partner_case_id TEXT
            )
        """)
    with connect(provider) as db:
        db.execute("""
            CREATE TABLE cases (
                key TEXT PRIMARY KEY NOT NULL,
                payload_json TEXT NOT NULL,
                case_id TEXT UNIQUE NOT NULL
            )
        """)


def save_intent(local, intent_id, payload):
    text = canonical(payload)
    with connect(local) as db:
        db.execute(
            "INSERT INTO intents VALUES (?, ?, ?, ?, ?, ?, 'pending', NULL)",
            (intent_id, payload["local_case_id"], VERSION, text,
             digest(text), f"partner-case:{intent_id}"),
        )


def read_intent(local, intent_id):
    with connect(local) as db:
        db.row_factory = sqlite3.Row
        row = db.execute(
            "SELECT * FROM intents WHERE intent_id=?", (intent_id,)
        ).fetchone()
        if row is None:
            raise LookupError(intent_id)
        return dict(row)


def replay_material(local, intent_id, current_payload):
    record = read_intent(local, intent_id)
```

PYTHON · 续

```
    if digest(canonical(current_payload)) != record["payload_digest"]:
        raise Conflict("input_changed_requires_new_intent")
    text = record["canonical_payload_json"]
    if record["payload_contract_version"] != VERSION or text is None:
        raise ValueError("pending_payload_not_replayable")
    if digest(text) != record["payload_digest"]:
        raise ValueError("saved_payload_inconsistent")
    return record["idempotency_key"], json.loads(text)


def create_case(provider, key, payload, calls, lose_response=False):
    calls.append(key)
    text = canonical(payload)
    with connect(provider) as db:
        db.execute("BEGIN IMMEDIATE")
        existing = db.execute(
            "SELECT payload_json, case_id FROM cases WHERE key=?", (key,)
        ).fetchone()
        if existing is not None:
            if existing[0] != text:
                raise Conflict("same_key_different_payload")
            return existing[1]
        number = db.execute("SELECT COUNT(*) FROM cases").fetchone()[0]
        case_id = f"P-{42 + number:04d}"
        db.execute("INSERT INTO cases VALUES (?, ?, ?)", (key, text, case_id))
    # 此处已经离开事务，新增案件已经提交。
    if lose_response:
        raise ResponseLost()
    return case_id


def confirm(local, intent_id, case_id):
    with connect(local) as db:
        changed = db.execute("""
            UPDATE intents SET state='partner_confirmed', partner_case_id=?
            WHERE intent_id=? AND state='pending'
        """, (case_id, intent_id)).rowcount
        if changed != 1:
            raise ValueError("pending_intent_required")


def effects(provider):
    with connect(provider) as db:
        return db.execute("SELECT COUNT(*) FROM cases").fetchone()[0]
```

PYTHON · 续

```
def main():
    payload = {
        "local_case_id": "LOCAL-0042", "category": "hardware",
        "summary": "Synthetic printer offline", "priority": 2,
    }
    calls = []
    with TemporaryDirectory(prefix="d03-intent-") as folder:
        local = Path(folder) / "local.sqlite3"
        provider = Path(folder) / "provider.sqlite3"
        setup(local, provider)
        save_intent(local, "OUT-0042-V1", payload)
        key, frozen = replay_material(local, "OUT-0042-V1", payload)
        try:
            create_case(provider, key, frozen, calls, lose_response=True)
        except ResponseLost:
            pass
        else:
            raise AssertionError("planned response loss did not occur")
        assert read_intent(local, "OUT-0042-V1")["state"] == "pending"
        assert len(calls) == 1 and effects(provider) == 1

        key, frozen = replay_material(local, "OUT-0042-V1", payload)
        case_id = create_case(provider, key, frozen, calls)
        confirm(local, "OUT-0042-V1", case_id)
        assert len(calls) == 2 and effects(provider) == 1
        assert calls[0] == calls[1] and case_id == "P-0042"
        print("recovered:", len(calls), effects(provider),
              read_intent(local, "OUT-0042-V1")["state"], case_id)

        changed = dict(payload, summary="Synthetic account access problem")
        try:
            replay_material(local, "OUT-0042-V1", changed)
        except Conflict:
            pass
        else:
            raise AssertionError("changed input was not rejected")
        assert len(calls) == 2
        try:
            create_case(provider, key, changed, calls)
        except Conflict:
            pass
        else:
            raise AssertionError("provider conflict was not exposed")
        assert len(calls) == 3 and effects(provider) == 1

        save_intent(local, "OUT-0042-V2", changed)
        new_key, new_frozen = replay_material(local, "OUT-0042-V2", changed)
```

PYTHON · 续

```
        new_id = create_case(provider, new_key, new_frozen, calls)
        confirm(local, "OUT-0042-V2", new_id)
        assert new_key != key and effects(provider) == 2
        assert read_intent(local, "OUT-0042-V1")["partner_case_id"] == "P-0042"

        save_intent(local, "OUT-0042-LEGACY", payload)
        with connect(local) as db:
            db.execute("""
                UPDATE intents
                SET canonical_payload_json=NULL, payload_contract_version=NULL
                WHERE intent_id='OUT-0042-LEGACY'
            """)
        before = len(calls)
        try:
            replay_material(local, "OUT-0042-LEGACY", payload)
        except ValueError as error:
            assert str(error) == "pending_payload_not_replayable"
        else:
            raise AssertionError("incomplete pending was replayed")
        assert len(calls) == before
        print("D03: persisted intent, conflict and replay boundaries passed")


if __name__ == "__main__":
    main()
```

运行 `python d03_idempotency_demo.py`，第一行应是：

```
recovered: 2 1 partner_confirmed P-0042
```

它记录最重要的恢复时刻。随后程序又故意直接向 provider 发了一次冲突请求，所以调用增加、效果仍为 1；再建立新的 V2 意图，效果才增加为 2。最后制造缺少输入的旧 pending，确认它在本地就被挡住，没有发出调用。这些额外场景不应混入前一行的“两次调用、一次效果”。

先核对第一行，再按顺序读后面的断言：`2` 是两次传输，`1` 是一次业务效果，`partner_confirmed` 是本地第二笔确认已经提交。后面的冲突和旧 pending 是另外两条停止路线；它们说明“调用次数增加”与“效果增加”不是同一个量，也说明没有重放材料时程序应保持未知，而不是猜测远端状态。

connect 管理器在正常离开时提交、异常时回滚，finally 再关闭连接。每次读回都经过重新连接，因此成功不是只在原连接里暂时可见；最后所有连接关闭，临时目录才能可靠清理。`@contextmanager` 让含 yield 的函数可用于 with：yield 交出 db，with 内动作结束后回到它后面继续提交或处理异常。这段资源管理可以查用，真正要能解释的是提交、回滚与关闭分别发生在哪一步。

D04 | 不能确认时，怎样把事情接住

对 LOCAL-0042 的两次调用都超时了。允许的尝试次数已经用完，合作方也没有交回可用案件编号。程序此时仍要给用户一个结果，但这个结果只能说明本地真正知道的事实。

可以说“本地已接收，合作方尚未确认，需要核对”；不能说“维修已经安排好”，也不能断言“合作方肯定没有案件”。本章处理的就是自动路线停止以后，事情由谁接住。

未完成需要有自己的状态

本地已经保存 OUT-0042-V1 的完整意图，包括合同版本、冻结输入、摘要和 key，初始状态为 pending。程序只在拿到满足合同的合作方案件结果后，才写 partner_confirmed；无法自动确认时，改为 manual_review，同时建立人工接管记录。

取得确认的分支

pending
↓
取得可用合作方结果，且本地确认提交成功
↓
partner_confirmed

停止自动路线的分支

pending
↓
自动路线停止，且本地接管记录提交成功
↓
manual_review

pending 表示意图存在，但本地还没有最终结论；它可能是在调用中，也可能在程序中断后留存。manual_review 再多说明一件事：自动路线已经停止，并留下待核对入口。这两个状态都不是对方“肯定失败”的证据。

本例公开结果采用以下字段：

JSON

```
{
  "local_case_id": "LOCAL-0042",
  "state": "manual_review",
  "partner_confirmed": false,
  "partner_case_id": null,
  "manual_review_required": true,
  "reason_code": "provider_timeout"
}
```

null 表示本地没有可确认的 provider ID，不表示 provider 数据库必定没有案件。状态、布尔值、编号和 reason_code 要共同说同一件事；若 state 写 manual_review，却保留 partner_confirmed=true，消费者会得到互相冲突的指示。

什么证据才足以写“合作方已确认”?

假设当前 provider 合同要求成功 body 为:

```JSON
{"case_id": "P-0001", "status": "created"}
```

adapter 检查成功 HTTP 状态、JSON、案件编号格式以及 status 的允许值，随后把 case_id 交给本地，映射成 partner_case_id。这个编号来自当前 intent 的那次调用，调用使用已保存 key 与冻结输入。若某个 provider 另外要求回复回显 key，也应检查它；本册配套的回复没有这个字段，不能编造一次不存在的回显验证。

真正的确认还要经过本地短事务：将 pending 改成 partner_confirmed，保存 P-0001，提交成功后再向用户报告。只有日志打印“成功”，或内存里已经拿到 P-0001，都不能替代本地提交。

如果这一步存储失败，外部 P-0001 仍可能存在。本地应保留可恢复意图，暴露本地故障，后续用原 key 核对。既不能把本地写失败归成“供应商不可用”，也不能宣称人工记录已经保存。

同样进入人工，保留的原因为什么不同?

已有证据	可以判断什么	不应怎样处理
两次 timeout	本地没有得到可用回复	不以“没做过”为前提生成新 key
允许重试的 5xx 持续出现	收到了失败状态，自动机会耗尽	不无上限增加尝试
401 或 403	当前凭据或权限被拒绝	不把重复相同凭据当修复
200，但 case_id 改名	有回复，但当前内容合同不成立	不猜最相似字段填编号
合法回复晚于 deadline	外部结果到达已晚于本地接纳期限	不把停止接纳解释成远端没执行

其中“字段改名”的情况最容易制造假成功。fake provider 先创建 P-0001，再返回 `{"case_ref":"P-0001","state":"created"}`；本地只认识 case_id 和 status，所以不能确认。外部效果数是 1，本地仍是 manual_review。没有可用编号与没有产生效果，仍然是两件事。

reason_code 把这些事实归成稳定的本地类别，便于用户界面、日志和处理队列配合；不应把原始 response 或异常原文直接交出去。人工看到 provider_timeout，优先核对已有结果；看到 provider_auth_failure，先找配置与权限负责人。相同最终状态可以有不同恢复路径。

202 到底承诺了什么?

本地接口用 202 接收这项未能自动确认的工作，并明确返回 manual_review。它只表示本地保存了待处理记录，不承诺“后台很快会成功”。HTTP 202 的一般语义也不保证最终执行结果；具体下一步必须由应用合同交代。RFC 9110 第 15.3.3 节

因此保存本地状态与创建人工记录要在同一个本地事务中完成。否则可能已经显示“正在人工处理”，队列却没有任何工作项。

BEGIN

OUT-0042-V1: pending → manual_review，保存 reason_code

新增指向 OUT-0042-V1 的 open 接管记录

↓ COMMIT

才向用户返回已接收的未确认状态

这两步都属于本地数据库，所以能够同成或同败。provider 的创建不在这个事务里；本地 rollback 撤回接管记录，也不会删除合作方案件。

人工接管应让下一步可执行

在实际系统中，接管需要明确角色，例如有派单核对权限的 dispatch_reviewer。记录至少能找到原意图、关联失败处理、看到原因和发生时间。完整 key 与 canonical 输入保存在受控意图库，普通队列显示所需引用，不把客户全文或 token 复制一遍。

配套的人工表内部以 intent_id 标识工作项；简化的列表输出是:

JSON

```
{
  "local_case_id": "LOCAL-0042",
  "request_id": "R-D04-MANUAL",
  "reason_code": "provider_timeout",
  "created_at": "2026-08-29T12:00:00Z",
  "status": "open"
}
```

这个输出用于验证最小字段，不是一套可投入使用的人工系统。特别是同一 local_case_id 存在多个 intent 时，正式处理入口必须明确选中哪一个工作项；不能只取“案件最新 intent”就处理所有旧接管记录。配套在表内保留 intent_id，但没有实现带鉴权的人工处理页面、通知送达、分派与完成回写。

真实的下一步应沿原工作项进行：取得权限后读取原 intent；用 provider 提供的查询或核对入口查 key 是否已有结果；若有，校验后关联同一个案件；若明确没有且业务仍需要，按当前合同决定重放；若不能判断，保持未确认并升级给指定负责人。人工也不能用新 key 一键盲发，否则只是把重复副作用从自动程序搬到了人工按钮。

日志应记录 request_id、操作名、attempt、耗时、错误类别和必要状态。token、Authorization、完整客户摘要、连接串与原始异常不是普通队列的必需输入。白名单先决定哪些字段可以记录，再由这些字段解释决策，通常比把所有内容打印后尝试遮盖更容易验证。OWASP Logging Cheat Sheet

哪些情况下可以给一个替代结果？

实时容量接口暂不可用，但本地有“09:00 观察到 2 个名额”。如果当前页面只是供工作人员估计繁忙程度，业务允许参考旧值，可以同时显示来源、观察时间以及“非实时，仅供参考”。这是一种受限 fallback。

它没有确认现在仍有 2 个名额，更没有创建任何案件。所以不能把这个旧容量转成 LOCAL-0042 的 partner_confirmed。来源是容量读取，需求是本次创建结果，两者根本不是同一个事实。

当前任务	可接受的出口
展示大致繁忙程度	业务允许时，显示带时间和限制的旧容量
当前凭据无效	清楚说明不可用，由配置或权限负责人处理
创建结果未知	保留未确认，核对已有案件或交给有权限的人
身份审批、付款确认等依赖可靠证据的决定	缺少证据就停止，不用猜测或旧值批准

clear failure、fallback、human handoff 不是三个互斥的“系统等级”。页面可以给受限参考，同时把创建动作交人工；高影响动作可以明确停止，同时提供下一步联系入口。选择取决于这个结果被拿来做什么，不能只看有没有一个值可以显示。

判断本章最重要的边界，只需看这一行：外部效果 1，本地 manual_review，公开编号 null。这可能完全正确，因为对方已创建但返回的内容无法校验。把它改成 partner_confirmed 需要新的成功证据；把外部效果改成 0 则需要对方未执行的证据。页面更好看不是其中任何一项。

D05 | 把调用、效果与本地状态放在一起验证

先沿一次成功和一次受控失败查看 provider 调用、本地状态、业务效果与人工出口分别留下什么证据。

一次创建可能先在合作方提交，随后超时，再用原 key 重传，最后本地确认。另一次可能收到 200，却因为字段变化而交人工。单独证明“会 retry”或“会转人工”，还不足以证明它们在同一份业务意图上协调正确。

配套 partner-case 将这些关系放进同一条路径。我们先看一份输入应留下哪些结果，再运行它；测试的重点是每个判断前后什么状态存在。

读 D05 时把入口和结果放在同一条 trace 上：请求先成为本地 pending，再经过有限的 provider 调用，最后只能落到 `partner_confirmed`、`manual_review` 或明确的冲突 / 拒绝。每个场景都从自己的新数据库开始；表中的调用数、效果数和本地结果是同一场景的三份证据，不能拿上一行的状态补这一行。

这次接进同一个程序的是什么？

本地入口为 `POST /support-cases`，接收：

JSON

```
{
  "local_case_id": "LOCAL-0042",
  "intent_id": "OUT-0042-V1",
  "category": "hardware",
  "summary": "Synthetic printer offline",
  "priority": 2
}
```

LOCAL-0042 是本地案件，OUT-0042-V1 是本次不可变外部动作。输入检查拒绝缺失字段、额外字段和不满足模型规则的值；priority 只允许规定的整数范围，不能用 urgent 这样的文字代替。CaseInput 的实际转换行为由固定 Pydantic 模型决定，并不因为使用模型就默认所有字段严格禁止转换。

本地意图保存 `payload_contract_version=partner-case-v1`、canonical_payload_json、SHA-256 payload_digest、稳定 key 和 pending。发往 provider 的业务 JSON 不包含 intent_id：它保留 local_case_id、category、summary、priority。intent 通过 `Idempotency-Key: partner-case:OUT-0042-V1` 关联；request ID 另经 X-Request-ID 关联当前一次处理。

合作方接口为 `POST /cases`，成功体要求 `{"case_id":"P-0001","status":"created"}`。这个 P-0001 由当前 fake 数据库生成，与 D03 独立程序中的 P-0042 不是同一行数据。adapter 校验后把 case_id 映射为本地的 partner_case_id。

> 本地 POST
> → FastAPI 输入模型
> → CaseService 保存或读取 intent
> → PartnerClient 执行一次有限调用策略
> → MockTransport 将 request 交给有状态 fake
> → fake 自己的 SQLite 保存 key 与业务结果
> → adapter 校验结果
> → 本地 SQLite 确认或建立人工接管
> → 本地公开 response

两个 SQLite 文件很关键：一个保存本地已知状态，另一个保存 fake provider 的真实效果。前者不知道的事情，可以已经存在于后者；把两者塞进同一份字典，会更难看出回复丢失的意义。

从回复丢失的一次创建开始核对

给 fake 设定 commit_then_timeout。首次本地输入先通过，完整 intent 提交为 pending，然后第一次调用才到达 fake。它创建 P-0001 并提交，接着抛出 ReadTimeout。本地没有取得 response，不能确认。

策略检查还有时间和次数，等待 0.2 秒，再用同一 key 和同一业务内容发送。fake 先按 key 查已存结果，找到原 P-0001，直接返回；它不再消耗一个“创建成功”的新剧本步骤。

本地校验回复后更新并提交，向用户返回 201、partner_confirmed 和 P-0001。这个时刻需要一起核对：

> provider_calls = 2
> provider_effects = 1
> 两次 key 都是 partner-case:OUT-0042-V1
> local_state = partner_confirmed
> partner_case_id = P-0001

第二次外部 response 是原结果，不代表发生第二次创建。本地 201 则表示这次提交路线确认了本地意图；不能把它与某次 provider 的 200/201 当成同一份 response。

随后用新的 request ID 再提交同一 intent 和同一内容，本地发现已经 confirmed，直接返回保存的结果，HTTP 为 200，provider 调用总数仍为 2。这里是本地读取挡住了外部调用；与“provider 收到重传后识别重复”是两道不同的保护。

换一个输入，应该改哪一份证据？

下表每个场景各用一份新数据库。重放与修改输入的场景先建立原意图，再执行相应动作；“上一行已完成后”明确沿用那一行的成功状态。调用和效果是该场景的总数，不把其他场景留下的记录累计进来。

场景	外部调用	外部效果	本地结果
合法输入，success	1	1	201，partner_confirmed
上一行已完成后，再提交同 intent/同内容	仍为 1	仍为 1	200，返回原结果
本地 priority 为 urgent	0	0	422，没有新 intent
status_500 后 success	2	1	201，partner_confirmed
连续 status_500	2	0	202，manual_review
timeout_before_commit 后 success	2	1	201，partner_confirmed
commit_then_timeout	2	1	201，同 key 取回原结果
success 消耗 2.6 秒后才返回	1	1	202，total_deadline_exhausted
status_401	1	0	202，provider_auth_failure
field_changed	1	1	202，provider_contract_changed
同 intent 被改成另一份输入	不新增调用	不新增效果	409，原记录保留
同案件的新意图、新输入、新 key	两个意图共 2	两个意图共 2	两条可追踪记录，各自确认
旧 pending 只有 digest，无版本或冻结输入	0	0	202，pending_payload_not_replayable

最后一行的效果 0 来自该实验已知的新 fake 基线。面对真实旧 pending，本地仅凭缺少输入不能推断对方效果为 0；只应停止自动重发并核对。

field_changed 先产生效果，再返回不符合 schema 的内容，故 1 次效果与 manual_review 可以同时成立。迟到成功也已经产生效果；本地因期限停止接纳，不等于取消了对方提交。

同案件新意图那一行，并不意味着换个 request ID 就创建第二次。改变的是明确的业务输入及 intent/key。旧 intent 被保留，新记录取得新的 creation_order；`GET /support-cases/LOCAL-0042` 按 `creation_order DESC` 读取最后建立的意图，不按时间文字或 intent 名称排序。GET 只读本地，未知案件返回 404；它不触发重试，也不删旧记录。

状态码和 reason_code 怎样交给下游？

本地 HTTP 结果	用户能据此知道什么
201 + partner_confirmed	本次处理已取得可用 provider 结果并完成本地确认
200 + partner_confirmed	已完成意图的本地结果直接重用
202 + manual_review	本地已保存人工接管，尚未确认合作方成功
409	同 intent 的输入冲突，不能覆盖原意图
422	输入在业务入口前失败，provider 必须零调用

GET 可以读到 pending，这意味着本地行已存在，但自动处理未留下最终结果。它不是另一种 confirmed。

需要排错时，再查具体 reason_code。普通阅读不必背这些字符串；应能由证据判断责任，然后按字段名找到代码和日志。

reason_code	找回的失败位置或停止原因
provider_timeout	没有可用 response，有限尝试未取得结果
provider_unavailable	合成合同允许重试的 5xx 用完机会，或其他传输不可用
provider_auth_failure	401/403，检查配置与权限，不重试
provider_idempotency_conflict	provider 409，原 key 与当前内容冲突
provider_contract_changed	成功状态后 JSON 或 schema 无法成为约定结果

reason_code	找回的失败位置或停止原因
provider_rejected	当前未单列的非成功状态，不猜测其可重试
retry_budget_exhausted	没有足够余额开始下一次或支付 backoff
total_deadline_exhausted	收到 response 时已经超过接纳期限
pending_payload_not_replayable	旧 pending 缺当前版本或完整冻结内容
provider_failure	防御性兜底，分类尚未明确，不能据此宣布成功

配套由 PartnerClient 一处负责 retry，本书默认场景最多尝试两次（含首次），实际次数上限读取配置。timeout 与 500/502/503/504 才按政策再次尝试。一般 RequestError 不自动重试，认证、冲突、拒绝和内容失败也停止。HTTPX 的阶段 timeout 与本地 deadline 分别检查；虚拟时钟故意允许模拟迟到，不代表真实调用已经被硬取消。

配置和秘密为什么也属于这条路线？

配置回答去哪、等多久、最多几次。token 只在需要构造 Authorization header 时展开。配套从 D_PARTNER_ 前缀读取环境设置，也允许测试显式提供假值：

```
D_PARTNER_BASE_URL=https://partner.invalid
D_PARTNER_TOKEN=<合成测试值>
D_PARTNER_TIMEOUT_SECONDS=1.0
D_PARTNER_TOTAL_BUDGET_SECONDS=2.5
D_PARTNER_MAX_ATTEMPTS=2
D_PARTNER_BACKOFF_SECONDS=0.2
```

测试会自己注入合成 token，不需要把真实凭据填进这份配置。Settings 只接受实验目标 partner.invalid；PartnerClient 必须显式收到 MockTransport，否则拒绝构造。trust_env=False 防止继承代理配置，follow_redirects=False 避免自动跟随另一目标。禁止真实 transport 才是主要的出站保护，示例域名本身不能替代它。

SecretStr 降低普通显示意外暴露的机会，但主动取出明文仍能打印。环境变量也不是保密仓库。程序不能把整个配置对象、原始 header、客户 summary 或原始异常写进普通事件；fake 只记录授权头是否存在，不保存头的值。白名单日志保留操作、attempt、耗时、status_code、错误类别等定位信息。Pydantic SecretStr

这是分清信息去向的练习：provider 请求需要的字段、本地重放需要的持久输入、用户 response、普通日志和人工列表，是五种不同的消费者，不应共享同一份“全部内容”。

先复制，再在实验目录运行

从资料包根目录的可见入口 `01_配套代码_A-E` 进入 `examples` → `d_dependency_reliability` → `partner-case`，把整个 `partner-case` 文件夹复制到自己选择的独立目录。这个入口与底层目录 `01_配套练习` 指向同一份配套文件。命名可以包含日期，以便保留不同实验；不要覆盖已有副本。所有下面的命令、章内脚本和修改都在副本中执行。

打开副本的 PowerShell，确认当前目录包含 requirements.txt 和 milestone_check.py。建立局部环境、安装固定依赖，再运行：

POWERSHELL

```
python --version
python -m venv .venv
if ($LASTEXITCODE -ne 0) { throw '创建环境失败' }
& .\.venv\Scripts\python.exe -m pip install -r .\requirements.txt
if ($LASTEXITCODE -ne 0) { throw '依赖安装失败' }
& .\.venv\Scripts\python.exe -m pip check
if ($LASTEXITCODE -ne 0) { throw '依赖关系不一致' }
& .\.venv\Scripts\python.exe -m pytest -q
if ($LASTEXITCODE -ne 0) { throw '测试失败，先查看对应详情' }
& .\.venv\Scripts\python.exe .\milestone_check.py
if ($LASTEXITCODE -ne 0) { throw '里程碑未通过' }
```

可使用 Python 3.13 运行这份固定配套；安装从自己配置的软件包来源获取依赖，实验本身只走 fake。若机器有多个 Python，先确认版本，再用选定解释器建立 .venv；运行时用上面显式的 .venv 路径，避免把依赖装给另一个解释器。

测试在系统临时目录建立两个数据库，已有同名文件会被拒绝，不会清空或覆盖。pytest 临时目录可能按保留策略留下便于排错；milestone_check 使用自己的 TemporaryDirectory，退出时关闭连接并清理，结果中的 temporary_files_removed 只说明这一轮里程碑自己创建的数据库已经移除。

里程碑成功应输出 status 为 PASS 的 JSON。查看各项字段，确认它们分别对应本地零调用、预算停止、两次传输一次效果、重建 fake 后识别原 key、坏 schema 不伪造成功、安全事件和临时状态清理。不能把最后一个 PASS 当成对真实供应商的可用性声明。

还要补看一个发生在两次提交之间的故障

外部成功与本地确认是两次提交。可以在实验副本新建 `d05_confirmation_recovery.py`，使用现有场景工具，故意让第一次本地 confirm_case 抛出 SQLite 错误。它导入的是同目录配套模块，不需要真实服务。

PYTHON

```
from pathlib import Path
from tempfile import TemporaryDirectory
from unittest.mock import patch
import sqlite3

from fake_partner import FakeStep
from milestone_check import scenario, valid_payload

with TemporaryDirectory(prefix="d05-recovery-") as folder:
    paths = []
    with scenario(
        Path(folder), "confirmation-recovery",
        [FakeStep("success")], paths
    ) as current:
        payload = valid_payload()
        with patch.object(
            current.store, "confirm_case",
            side_effect=sqlite3.OperationalError("synthetic local write
              failure")
        ):
            try:
                current.client.post(
                    "/support-cases", json=payload,
                    headers={"X-Request-ID": "R-LOCAL-WRITE-FAIL"}
                )
            except sqlite3.OperationalError:
                pass
            else:
                raise AssertionError("planned local failure did not occur")
        pending = current.store.get_case("LOCAL-0042")
        assert pending.state == "pending"
        assert pending.payload_contract_version == "partner-case-v1"
        assert pending.canonical_payload_json is not None
        assert pending.payload_digest
        assert current.fake.call_count == 1
        assert current.fake.business_effect_count == 1

        replay = current.client.post(
            "/support-cases", json=payload,
            headers={"X-Request-ID": "R-CONTROLLED-REPLAY"}
        )
```

PYTHON · 续

```
        assert replay.json()["state"] == "partner_confirmed"
        assert replay.json()["partner_case_id"] == "P-0001"
        assert current.fake.call_count == 2
        assert current.fake.business_effect_count == 1
        assert current.fake.idempotency_keys == [
            "partner-case:OUT-0042-V1", "partner-case:OUT-0042-V1"
        ]
assert all(not path.exists() for path in paths)
print("D05: local confirmation recovered with one provider effect")
```

用实验环境运行该脚本。第一次错误暴露给调用者，pending 仍在，provider 效果已经是 1。第二次是新的本地请求，但业务输入和 intent 未变；它使用完整冻结输入与原 key 取回 P-0001，才把本地状态确认。若只保存 digest 而没有 canonical 内容，恢复必须停在本地，不能走这条发送路线。

这个实验不是在证明存储系统从不失败，而是在证明：局部写入失败后仍有足够材料判断该恢复哪一件事。它没有演练真实断电、文件损坏或多进程竞争。

这段恢复脚本只需要抓住输入、故障点和下一次输出：第一次请求的 provider 已产生一个效果，但 `confirm_case` 被安排失败；第二次请求读取同一 intent 的冻结内容和原 key，取回同一 `P-0001`，然后才完成本地确认。看到 `pending`、`business_effect_count == 1` 和最终 `partner_confirmed` 三个时点，才能把“本地写失败”与“远端未执行”区分开。

以后需要修改时，从哪个文件找起？

要理解或改变的关系	配套入口
本地字段与成功模型	models.py
先保存 intent，再调用，再确认或交接	app.py 的 CaseService
request、异常分类、预算与重试	partner_client.py、retry_policy.py
已提交后丢回复，按 key 取回原结果	fake_partner.py
本地确认、接管事务、最新意图查询	state.py
配置入口、秘密与事件去向	config.py、observability.py
某条分支实际断言了哪些事实	对应 test_d01 至 test_d05 文件及 milestone_check.py

修改时先挑一条关系。例如把最多尝试改为 1，就应预测 commit_then_timeout 留下一个外部效果、本地无法自动确认，而非预测效果变成 0。先写下这两个状态，再看运行结果，能暴露是否把“没有重试”误解成“没有执行”。

TestClient、MockTransport 和虚拟时钟让本地路线稳定可重复；它们不经过实际监听端口、DNS、TLS、供应商凭据或配额。临时 SQLite 也不证明所有并发与宕机恢复。人工列表证明记录已保存，不证明真实角色已获授权或已经接单。进入真实集成时，应围绕目标 provider 的接口、key 保留期限、错误许可和非生产环境补证据；本配套没有真实 provider 入口。

全书收束 | 外部依赖可靠性的知识关系总览

外部系统可能慢、出错、改变字段，或完成以后没有把回复送回来。本地无法控制这些事情始终不发生，但可以控制下一次是否继续、怎样避免重复、自己保存什么，以及向用户承诺什么。

现在可以把本册知识按它们保护的关系重新放在一起：

取得结果	本地 validation：不符合约定的输入先停在本地 adapter：构造外部请求，检查 status 和内容，交回明确结果 fake + MockTransport：重复制造外部返回与失败
决定下一步	timeout：限制某次等待 deadline + attempt limit：限制继续等待与调用的机会 backoff：下一次何时开始，仍花同一份时间预算 intent + 不可变输入 + key + 对方持久记录：重复是否安全
保存与表达	本地 transaction：完整保存意图、确认或人工接管 日志：关联某次处理的尝试与失败证据 fallback：提供来源、时间和用途明确的有限参考 manual_review：把未知结果交给有权限的下一处理者

这些工具不是沿着一条固定流水线各用一次。失败类别影响是否重试，重试又需要预算与幂等保护；内容失败影响本地确认，确认写入失败又会回到持久意图的恢复；人工也必须遵守相同的业务身份和权限。

最容易做错的判断，都在相邻责任之间

现场看见什么	应当找哪种关系	近邻工具为什么不能代替
本地 422，却仍发出外部调用	本地输入应先于业务执行	外部没有效果不等于零调用
合作方 500，本地 502	区分两次 HTTP 交换	最后一个状态码不能表示整个经过

现场看见什么	应当找哪种关系	近邻工具为什么不能代替
单次 timeout 一秒，整项等待很长	总 deadline、次数与 backoff	单次 timeout 不计算其他尝试
401、坏字段反复出现	判断错误是否有理由再次尝试	更多 retry 不能修复凭据或解释字段
timeout 后不敢再次创建	稳定 intent/key 与对方结果记录	request ID 和“已发送”日志不能去重
旧 pending 只剩一个摘要	完整可重放输入与合同版本	摘要不能还原请求
页面显示已创建，但没有可用编号	结果合同和本地确认提交	HTTP 200、旧容量或猜测不能补出事实
队列写了人工处理，却无人能继续	工作项身份、权限与下一动作	manual_review 名称不能代替真实接管

选择工具时先问缺了哪份证据。例如已经拿到合法 provider ID，但本地更新失败，应看本地提交和原 intent 恢复；不能把所有失败都交给网络 retry。反过来，本地事务完整也不能证明远端没有重复创建。

同一个案件，为什么需要几个不同编号？

LOCAL-0042 指出业务对象。OUT-0042-V1 指出为它发起的这一次外部动作，原 key 绑定该动作。R-501 或 R-502 指出先后两次本地处理，供日志关联。P-0001 指出合作方最终保存的结果。

重复处理同一意图时，request ID 可以变化，intent/key 和冻结输入不变。真实业务内容变化时，新建 intent/key，旧记录保留。provider 的结果编号只能通过它的合同取得，不从 local_case_id 的数字推算。

这也解释为何公开工单 T-42 不会自动变成 LOCAL-0042：两者属于不同示例和标识空间。跨册连接的是责任与数据流，不是默认共享一份数据库。

用一次失败把整条关系闭合

从尚未处理的 LOCAL-0042 开始，本地先把 OUT-0042-V1、partner-case-v1、canonical 输入、SHA-256 摘要、key 和 pending 一起提交，再开始网络调用。

第一次 request 到达 provider，它保存 key 与结果 P-0001，提交后丢失回复。本地得到 timeout，只知道这次没有取得可用结果。策略检查这个 provider 的重放合同、次数与剩余预算；还允许继续时，第二次仍读取原输入和 key。provider 返回 P-0001，adapter 校验字段，本地确认提交后才向用户公开成功。

这里最有解释力的结果是“2 次调用，1 次效果，partner_confirmed”。如果第二次仍未能得到可用结果，则停止自动路线，保存 manual_review 和接管工作项；provider 效果仍可能是 1。没有得到成功回复，不能把已有效果擦成 0。

如果程序在第一次调用后退出，后来另一请求遇到 pending，必须先恢复完整输入和合同版本，并核对当前业务内容。如果资料不足，不能由摘要猜回请求；如果内容已变，不能继续拿旧 key 表示新意图。时间预算通常属于当前一次处理，业务身份却需要跨处理保留，这是两个不同寿命的关系。

什么需要真正理解，什么可以遇到时查？

应当能独立解释四件事：失败最早在哪；当前证据是否足以再次传输；重新传输如何保持同一意图；最后哪些状态允许写成成功。它们会在支付、通知、派单、身份核验和模型调用中反复出现，不能只依赖看见代码才想起。

构造测试场景、查看调用数和效果数、沿 request ID 找日志、确认停止条件，则会在实际运行中逐渐用顺。每个 provider 的精确 header、key 有效期、429/5xx 许可、SDK 参数以及配置名称，适合在接入时查官方合同，并用独立实验验证。记住别家服务的 retry 清单，不能替代这一步。

复杂平台暂时没有成为本册的前提。只有已经出现可复现的并发重复，才进一步研究竞争与唯一性；只有需要可靠异步投递，才研究队列和 outbox；只有持续故障正在耗尽共享资源，才研究熔断和隔离。新机制应回到原来的调用、效果、本地状态和恢复矩阵，证明它解决了实际问题。

模型调用会把哪条边界再往前推？

模型 provider 也可能超时、不可用、返回坏格式，仍需要本册的时间、秘密、重试和人工出口。但即使它返回 200，JSON 字段都正确，也可能没有事实依据或提出未获授权的动作。

E 册因此从“取得一个格式合法的外部结果”继续，区分模型提出的内容、应用认可的用途、资料依据、工具权限和人工确认。D 册的 partner_confirmed 与 E 册的 needs_human 等状态属于不同对象，不能只因都表示“结果”就混在一起。

来源与查用入口

本册的业务输入、fake、状态名与时间数字为自包含实验合同。协议与工具行为可以从下面官方入口找回：

- RFC 9110：HTTP Semantics：安全、幂等方法、自动重试与 202 语义，属于固定协议文本。

- HTTPX 的 Timeouts、Exceptions 和 Transports：阶段等待、异常层级与替换传输。
- FastAPI 的 Testing 与 Handling Errors：进程内验证与公开错误处理。
- Python 的 time.monotonic 和 sqlite3：经过时间与本地持久状态。
- Pydantic 的 Settings 与 SecretStr：环境入口与普通显示遮盖。
- OWASP 的 Logging Cheat Sheet 和 Secrets Management Cheat Sheet：记录范围与秘密管理。
- AWS Builders' Library 的 Timeouts, retries and backoff with jitter：重试放大、等待与副作用的工程实践。

除固定 RFC 外，上述网页是持续更新的 live 文档，升级或真实接入时需要重新核对。固定依赖文件用于复现配套，不表示其中版本永久最新。`Idempotency-Key` 在本册是 fake 明确实现的 header 合同；无论相关标准化工作处于什么阶段，都不能由这个名称推定任意目标 provider 已支持相同的保存期限、冲突规则或恢复能力。

本册总览（需要时再回看）

这里保留整册地图、章节关系和查用入口；第一次阅读不需要先读完。

本册知识结构总览

先看几个相互配合的问题。章节顺序是理解顺序；下图中的分组不是互相隔绝的程序层。

一个步骤需要另一套系统完成

回来的东西能不能使用？	输入是否符合约定、回复是否收到且内容合用 D01 用可控替身重复制造这些结果 D01
没得到可用结果，下一次还能做什么？	时间和次数是否还够 D02 再传一次，会不会再做一遍 D03 无法安全继续时，怎样说明未完成、交给人 D04
怎样证明这些决定在同一程序里成立？	分别核对调用、外部效果、本地状态和运行线索 D05

限制等待与避免重复，是两种会同时需要的保护。等待时间很短的请求也可能重复创建；不会重复创建的请求也可能一直占用资源。把二者结合起来，才有理由决定是否再次尝试。人工接管接住的是这组自动决定无法安全完成的部分，并不把未知结果改成成功。

容易混在一起的两件事	本册怎样分开
收到 HTTP 回复与业务已完成	状态码后面还要检查内容，并确认它能代表当前动作
超时与对方没做	超时描述调用者没有及时取得结果，对方可能已经执行
再次发送与再次产生效果	同一意图可以多次传输，但需要对方识别重复并返回原结果
本地事务与远端事务	本地撤回不能自动撤回另一套系统的提交
测试通过与真实供应商可用	可控测试证明指定情形下的本地行为，真实环境另有证据

第一次阅读不必背这张表。先记住实际追踪的三份事实：**调用了几次，对方做了几次，本地最后保存了什么。** 后面的具体例子会让这些差别逐一可见。

这一册接在数据库的什么位置？

本地已经能保存业务对象和处理记录。现在其中一步要交给合作方：

本地输入 → 保存待执行意图 → 调用合作方
合作方拥有自己的持久数据
→ 检查结果 → 更新本地状态

本地数据库与合作方数据库不是一个事务。请求在两边之间流动，事实由两边分别保存。程序必须允许一种暂时不一致：对方已有结果，本地还不能确认它。

本册的本地案件写作 `LOCAL-0042`，问题是打印机离线。它不是公开工单号 `T-42` 的另一种拼法；不同示例程序没有自动共享数据库。稍后出现的 `OUT-*` 表示为本地案件发起的一次外部动作，`P-*` 表示合作方产生的结果。D 与 E 即使沿用 LOCAL-0042，配套程序的状态也各自独立。

前两章只做读取：查询邮编服务范围、查询某日容量。它们让我们先看清失败和预算，不提前假设写请求可以随意重试。第三章才创建案件，第四章处理未确认的出口，第五章把各项关系放进一个可运行程序。

实验怎样保持可重复？

本册所有程序都使用合成数据与可控替身，不连接真实 provider。provider 是提供外部能力的系统；fake 是模拟它行为的本地程序。fake 可以准确地“创建完结果再丢回复”，让同一条失败反复出现。

读到完整代码时，先沿输入预测结果，再运行核对。需要熟悉的是判断失败位置、计算剩余预算、识别稳定业务意图和读回状态；整段环境配置与字段拼写可以查用。仅记得 timeout、retry 等名称，还不足以判断一次创建能否再次发送。

所有需要保存脚本、建环境或修改状态的操作，先从资料包根目录的 `01_配套代码_A-E` 进入 `examples → d_dependency_reliability`，把 `partner-case` 整个复制到自选的独立实验目录。后文新增的章内脚本也保存在这个实验副本中。参考配套保留原样，实验可以随时从新副本重做；程序自己的数据库另在系统临时目录创建。